Premier League Model Upgrade Guide

Oct 5, 2026 · @Kyan

Overview

Can you do all of them? Yes. Each upgrade builds on the one before, and the guide is written so you can work straight through it. Doing one upgrade per session (an evening or a weekend afternoon each) is realistic. The whole guide is roughly 15–25 hours of work, with Upgrade 6 (Python) the longest.

What you'll end up with: a model that adjusts for home advantage, doesn't overreact early in the season, handles promoted teams with a statistical method, uses expected goals, and is benchmarked against the bookmakers and a professional-grade Dixon–Coles model, with every change backed by measured results.

The order, and why

Step	Upgrade	Time	Tools	Why at this point
Stage 0	Measurement foundation (includes RPS)	1–2 h	SQL, Excel	Every later upgrade is tested with it
1	Home advantage	1–2 h	SQL, Excel	Biggest, simplest gain; your calibration chart already shows the gap
2	Blend ratings by games played	1–2 h	SQL, Excel	Fixes early-season overreaction (Brighton 56% after 5 games)
3	Promoted-team regression	2 h	SQL, Excel	Your weakest forecast is pre-season
4	Bookmaker benchmark	2–3 h	Excel, SQL	The professional yardstick
5	Expected goals (xG) ratings	3–4 h	Excel, SQL	More stable ratings from fewer games
6	Dixon–Coles model in Python	4–8 h	Python, SQL	The academic standard; biggest CV upgrade

The golden rules

1. **Test in SQL first, change Excel second.** Each upgrade is first added to the database as a *model variant*: a named set of settings (e.g. `home_adv`) that SQL uses to re-predict every past match. You compare it with the current model on the same matches. Only if it wins do you change your Excel model.
2. **One change at a time.** If you change two things at once, you can't tell which one helped.
3. **Never load two runs for the same gameweek.** Your dashboard charts add up values per gameweek, so two `after GW6` runs would double the bars. Variants are tested in SQL, which avoids this.
4. **Bigger samples beat smaller ones.** A result on 50 matches can easily be luck. Trust results on hundreds of matches, and prefer the RPS score, the standard for football.
5. **Back up before each upgrade.** Copy your 2026-27 model workbook and `Prem_Dashboard.xlsx` into a `backup` folder first.
6. **Write down each result** in the log below. It becomes your CV evidence.

How to judge a result

- **Brier score** and **RPS (Ranked Probability Score):** lower is better. RPS is the standard in published football research; good models score around 0.19–0.21.
- A drop of **0.002 RPS or more on 300+ matches** is a meaningful improvement. Smaller changes, or changes on fewer matches, may be noise.
- Keep the change if it improves RPS on the largest sample available, even if a small sample disagrees.

Results log (fill in as you go)

Upgrade	Variant name	Run tested	Matches	RPS before	RPS after	Kept?
1 Home advantage	home_adv	2025-26 preseason				
2 Blending	blend_10	2025-26 after GW22				
3 Promoted teams	promoted_reg	preseason runs				
4 Bookmakers	bookmaker	same matches as model				(benchmark)
5 xG	xg_50	after-GW runs				
6 Dixon–Coles	dixon_coles	2026-27 runs				

Files you'll need

- `dc_model.py`: the Python script for Upgrade 6 (attached in the chat).
- Your 2026-27 model workbook (`prem_simulation_26-27.xlsx`) and `Prem_Dashboard.xlsx`.
- pgAdmin, connected to `prem_sims`.
- A new SQL file for each stage, saved in your `sql` folder with the names given.

Stage 0: Measurement foundation

This stage builds the testing system every upgrade uses. It adds no new model yet: at the end, your scores are exactly the same as now, plus the RPS score and a comparison page in Excel.

What gets built

Object	Type	Purpose
`model_variants`	table	One row per variant: home/away factors, blend weight, xG weight, promoted-team switch, and which variant is **adopted** (used by your scorecard)
`match_xg`, `promoted_inputs`, `external_predictions`, `dc_sim_summary`, `dc_team_ratings`	tables	Empty for now; filled in Upgrades 3–6
`v_run_avg`, `v_team_games`, `v_xg_ratings`, `v_promoted_fit`	views	Helpers: average goals per run, games played per team, xG ratings, promoted-team regression
`v_variant_ratings`	view	Each team's attack/defence for every run **and** every variant
`v_variant_predictions`	view	Home/draw/away probabilities for every fixture, run and variant
`v_match_predictions`, `v_brier`, `v_calibration`	views	Rebuilt on top of the above; they now follow the **adopted** variant, so your Model scorecard keeps working unchanged
`v_all_predictions`, `v_scored_predictions`, `v_scores`, `v_variant_calibration`	views	Every model and external source scored with Brier **and** RPS

Step 0.1: Back up

Copy `Prem_Dashboard.xlsx` and your 2026-27 model workbook into a `backup` folder.

Step 0.2: Run the foundation script

Open pgAdmin → click **prem_sims** → **Tools** → **Query Tool**. Paste the whole script, press **F5 with nothing highlighted**, and save it as `sql\11_foundation.sql`.

```
-- ===== 1. New tables =====
CREATE TABLE IF NOT EXISTS model_variants (
  variant      TEXT PRIMARY KEY,
  description  TEXT,
  home_factor  NUMERIC NOT NULL DEFAULT 1,
  away_factor  NUMERIC NOT NULL DEFAULT 1,
```

```
  blend_k      NUMERIC,                    -- NULL = no blending
  xg_weight    NUMERIC NOT NULL DEFAULT 0, -- 0 = goals only, 1 = xG only
  promoted_reg BOOLEAN NOT NULL DEFAULT FALSE,
  adopted      BOOLEAN NOT NULL DEFAULT FALSE
);
INSERT INTO model_variants (variant, description, adopted)
VALUES ('base', 'Original model: run ratings, no home advantage', TRUE)
ON CONFLICT (variant) DO NOTHING;

CREATE TABLE IF NOT EXISTS match_xg (
  match_id INT PRIMARY KEY REFERENCES matches(match_id),
  xg_home NUMERIC, xg_away NUMERIC
);
CREATE TABLE IF NOT EXISTS promoted_inputs (
  season_label TEXT, team_id INT REFERENCES teams(team_id),
  champ_att NUMERIC, champ_def NUMERIC,
  PRIMARY KEY (season_label, team_id)
);
CREATE TABLE IF NOT EXISTS external_predictions (
  source TEXT, run_label TEXT, match_id INT REFERENCES matches(match_id),
  p_home NUMERIC, p_draw NUMERIC, p_away NUMERIC,
  PRIMARY KEY (source, run_label, match_id)
);
CREATE TABLE IF NOT EXISTS dc_sim_summary (
  run_label TEXT, team_name TEXT, avg_finish NUMERIC,
  title_pct NUMERIC, top4_pct NUMERIC, relegation_pct NUMERIC
);
CREATE TABLE IF NOT EXISTS dc_team_ratings (
  run_label TEXT, team_name TEXT, attack NUMERIC, defence NUMERIC,
  home_adv NUMERIC, rho NUMERIC, avg_goals NUMERIC
);

-- ===== 2. Remove views that are rebuilt below =====
DROP VIEW IF EXISTS v_vs_bookmaker, v_promoted_check,
                    v_variant_calibration, v_scores, v_scored_predictions,
v_all_predictions,
                    v_calibration, v_brier, v_rps, v_match_predictions,
                    v_variant_predictions, v_variant_ratings, v_promoted_fit,
                    v_xg_ratings, v_team_games, v_run_avg;

-- ===== 3. Helper views =====
CREATE VIEW v_run_avg AS
SELECT r.run_id, r.run_label, r.season_id, r.season, r.gameweek,
       CASE WHEN r.gameweek = 0 THEN
            (SELECT SUM(f.gf)::NUMERIC / SUM(f.played)
             FROM final_tables f JOIN seasons s2 USING (season_id)
```

```
             WHERE s2.label < r.season
             GROUP BY s2.label ORDER BY s2.label DESC LIMIT 1)
       ELSE
            (SELECT SUM(m.home_goals + m.away_goals)::NUMERIC / (2 *
COUNT(*))
             FROM matches m
             WHERE m.season_id = r.season_id AND m.round_no <= r.gameweek
               AND m.home_goals IS NOT NULL)
       END AS avg_goals
FROM v_runs r;

CREATE VIEW v_team_games AS
SELECT r.run_id, t.team_id, COUNT(m.match_id) AS games
FROM v_runs r
CROSS JOIN teams t
LEFT JOIN matches m
       ON m.season_id = r.season_id AND m.round_no <= r.gameweek
      AND m.home_goals IS NOT NULL
      AND (m.home_team_id = t.team_id OR m.away_team_id = t.team_id)
GROUP BY r.run_id, t.team_id;

CREATE VIEW v_xg_ratings AS
WITH tm AS (
  SELECT r.run_id, m.home_team_id AS team_id, x.xg_home AS xgf, x.xg_away AS
xga
  FROM v_runs r
  JOIN matches m  ON m.season_id = r.season_id AND m.round_no <= r.gameweek
AND m.home_goals IS NOT NULL
  JOIN match_xg x ON x.match_id = m.match_id
  UNION ALL
  SELECT r.run_id, m.away_team_id, x.xg_away, x.xg_home
  FROM v_runs r
  JOIN matches m  ON m.season_id = r.season_id AND m.round_no <= r.gameweek
AND m.home_goals IS NOT NULL
  JOIN match_xg x ON x.match_id = m.match_id
),
lg AS (SELECT run_id, AVG(xgf) AS lg_avg FROM tm GROUP BY run_id)
SELECT tm.run_id, tm.team_id, COUNT(*) AS xg_games,
       1 + (AVG(tm.xgf) / lg.lg_avg - 1) * 0.75 AS att_xg,
       1 + (AVG(tm.xga) / lg.lg_avg - 1) * 0.75 AS def_xg
FROM tm JOIN lg USING (run_id)
GROUP BY tm.run_id, tm.team_id, lg.lg_avg;

CREATE VIEW v_promoted_fit AS
SELECT COUNT(*) AS clubs,
       REGR_SLOPE(epl_att, champ_att)     AS att_slope,
```

```
       REGR_INTERCEPT(epl_att, champ_att) AS att_intercept,
       CORR(epl_att, champ_att)           AS att_corr,
       REGR_SLOPE(epl_def, champ_def)     AS def_slope,
       REGR_INTERCEPT(epl_def, champ_def) AS def_intercept,
       CORR(epl_def, champ_def)           AS def_corr
FROM promoted_history;

-- ===== 4. Ratings for every run and every variant =====
CREATE VIEW v_variant_ratings AS
WITH base AS (
  SELECT v.variant, v.blend_k, v.xg_weight, v.promoted_reg,
         r.run_id, r.season, r.gameweek, tr.team_id,
         tr.att, tr.def,
         pre.att AS pre_att, pre.def AS pre_def,
         COALESCE(g.games, 0) AS games,
         xr.att_xg, xr.def_xg
  FROM model_variants v
  CROSS JOIN v_runs r
  JOIN team_ratings tr       ON tr.run_id = r.run_id
  LEFT JOIN v_runs rp        ON rp.season_id = r.season_id AND rp.gameweek =
0
  LEFT JOIN team_ratings pre ON pre.run_id = rp.run_id AND pre.team_id =
tr.team_id
  LEFT JOIN v_team_games g   ON g.run_id = r.run_id AND g.team_id =
tr.team_id
  LEFT JOIN v_xg_ratings xr  ON xr.run_id = r.run_id AND xr.team_id =
tr.team_id
),
blended AS (
  SELECT b.*,
         CASE WHEN b.blend_k IS NOT NULL AND b.gameweek > 0 AND b.pre_att IS
NOT NULL
              THEN (b.pre_att * b.blend_k + b.att * b.games) / (b.blend_k +
b.games)
              ELSE b.att END AS att_b,
         CASE WHEN b.blend_k IS NOT NULL AND b.gameweek > 0 AND b.pre_def IS
NOT NULL
              THEN (b.pre_def * b.blend_k + b.def * b.games) / (b.blend_k +
b.games)
              ELSE b.def END AS def_b
  FROM base b
),
promoted AS (
  SELECT bl.*,
         CASE WHEN bl.promoted_reg AND bl.gameweek = 0 AND pi.team_id IS NOT
NULL
```

```
              THEN (f.att_slope * pi.champ_att + f.att_intercept)::NUMERIC
              ELSE bl.att_b END AS att_p,
         CASE WHEN bl.promoted_reg AND bl.gameweek = 0 AND pi.team_id IS NOT
NULL
              THEN (f.def_slope * pi.champ_def + f.def_intercept)::NUMERIC
              ELSE bl.def_b END AS def_p
  FROM blended bl
  LEFT JOIN promoted_inputs pi ON pi.season_label = bl.season AND pi.team_id
= bl.team_id
  CROSS JOIN v_promoted_fit f
)
SELECT variant, run_id, team_id,
       CASE WHEN xg_weight > 0 AND att_xg IS NOT NULL
            THEN (1 - xg_weight) * att_p + xg_weight * att_xg ELSE att_p END
AS att,
       CASE WHEN xg_weight > 0 AND def_xg IS NOT NULL
            THEN (1 - xg_weight) * def_p + xg_weight * def_xg ELSE def_p END
AS def
FROM promoted;

-- ===== 5. Predictions for every fixture, run and variant =====
CREATE VIEW v_variant_predictions AS
WITH lambdas AS (
  SELECT vh.variant, ra.run_id, ra.run_label, ra.season, ra.gameweek,
         m.match_id, m.round_no, m.match_date,
         th.team_name AS home_team, ta.team_name AS away_team,
         m.home_goals, m.away_goals,
         vh.att * va.def * ra.avg_goals * v.home_factor AS xg_home,
         va.att * vh.def * ra.avg_goals * v.away_factor AS xg_away
  FROM v_run_avg ra
  JOIN matches m            ON m.season_id = ra.season_id
  JOIN v_variant_ratings vh ON vh.run_id = ra.run_id AND vh.team_id =
m.home_team_id
  JOIN v_variant_ratings va ON va.run_id = ra.run_id AND va.team_id =
m.away_team_id
                           AND va.variant = vh.variant
  JOIN model_variants v     ON v.variant = vh.variant
  JOIN teams th             ON th.team_id = m.home_team_id
  JOIN teams ta             ON ta.team_id = m.away_team_id
),
grid AS (
  SELECT l.variant, l.run_id, l.match_id, g.hg, g.ag,
         EXP(-l.xg_home::FLOAT8) * POWER(l.xg_home::FLOAT8, g.hg) /
FACTORIAL(g.hg)::FLOAT8
       * EXP(-l.xg_away::FLOAT8) * POWER(l.xg_away::FLOAT8, g.ag) /
FACTORIAL(g.ag)::FLOAT8 AS p
```

```
  FROM lambdas l
  CROSS JOIN (SELECT hg, ag FROM generate_series(0, 10) AS hg,
generate_series(0, 10) AS ag) g
),
probs AS (
  SELECT variant, run_id, match_id,
         SUM(p) FILTER (WHERE hg > ag) / SUM(p) AS p_home,
         SUM(p) FILTER (WHERE hg = ag) / SUM(p) AS p_draw,
         SUM(p) FILTER (WHERE hg < ag) / SUM(p) AS p_away
  FROM grid
  GROUP BY variant, run_id, match_id
)
SELECT l.variant, l.season, l.run_label, l.gameweek AS run_gameweek,
       l.round_no, l.match_date, l.home_team, l.away_team,
       ROUND(l.xg_home, 2) AS xg_home, ROUND(l.xg_away, 2) AS xg_away,
       ROUND(p.p_home::NUMERIC, 4) AS p_home,
       ROUND(p.p_draw::NUMERIC, 4) AS p_draw,
       ROUND(p.p_away::NUMERIC, 4) AS p_away,
       l.home_goals, l.away_goals,
       CASE WHEN l.home_goals IS NULL THEN NULL
            WHEN l.home_goals > l.away_goals THEN 'H'
            WHEN l.home_goals = l.away_goals THEN 'D'
            ELSE 'A' END AS result,
       (l.round_no > l.gameweek) AS out_of_sample,
       l.run_id, l.match_id
FROM lambdas l
JOIN probs p ON p.variant = l.variant AND p.run_id = l.run_id AND p.match_id
= l.match_id;

-- ===== 6. Your existing views, now following the adopted variant =====
CREATE VIEW v_match_predictions AS
SELECT season, run_label, run_gameweek, round_no, match_date, home_team,
away_team,
       xg_home, xg_away, p_home, p_draw, p_away, home_goals, away_goals,
       result, out_of_sample, run_id, match_id
FROM v_variant_predictions
WHERE variant = COALESCE((SELECT variant FROM model_variants WHERE adopted
ORDER BY variant LIMIT 1), 'base');

CREATE VIEW v_brier AS
WITH s AS (
  SELECT run_label, season, run_gameweek,
         COUNT(*) AS matches_tested,
         AVG(POWER(p_home - (result = 'H')::INT, 2)
           + POWER(p_draw - (result = 'D')::INT, 2)
           + POWER(p_away - (result = 'A')::INT, 2)) AS brier_model,
```

```
         1 - (POWER(AVG((result = 'H')::INT), 2)
            + POWER(AVG((result = 'D')::INT), 2)
            + POWER(AVG((result = 'A')::INT), 2)) AS brier_baseline,
         AVG(CASE WHEN (CASE WHEN p_home >= p_draw AND p_home >= p_away THEN
'H'
                             WHEN p_away >= p_draw THEN 'A'
                             ELSE 'D' END) = result
                  THEN 1.0 ELSE 0 END) AS pct_correct_pick
  FROM v_match_predictions
  WHERE result IS NOT NULL AND out_of_sample
  GROUP BY run_label, season, run_gameweek
)
SELECT run_label, season, run_gameweek, matches_tested,
       ROUND(brier_model, 4)    AS brier_model,
       ROUND(brier_baseline, 4) AS brier_baseline,
       ROUND(100 * (1 - brier_model / NULLIF(brier_baseline, 0)), 1) AS
improvement_pct,
       ROUND(pct_correct_pick, 3) AS pct_correct_pick
FROM s;

CREATE VIEW v_calibration AS
SELECT run_label, season,
       FLOOR(p_home * 10) / 10 AS band_start,
       (FLOOR(p_home * 10) * 10)::INT || '-' || (FLOOR(p_home * 10) * 10 +
10)::INT || '%' AS band,
       COUNT(*) AS matches,
       ROUND(AVG(p_home), 3) AS avg_predicted,
       ROUND(AVG((result = 'H')::INT), 3) AS actual_rate
FROM v_match_predictions
WHERE result IS NOT NULL AND out_of_sample
GROUP BY run_label, season, FLOOR(p_home * 10);

-- ===== 7. Every model and external source, scored =====
CREATE VIEW v_all_predictions AS
SELECT variant AS model, season, run_label, run_gameweek, match_id, round_no,
       p_home, p_draw, p_away, result, out_of_sample
FROM v_variant_predictions
UNION ALL
SELECT e.source, s.label, e.run_label, r.gameweek, e.match_id, m.round_no,
       e.p_home, e.p_draw, e.p_away,
       CASE WHEN m.home_goals IS NULL THEN NULL
            WHEN m.home_goals > m.away_goals THEN 'H'
            WHEN m.home_goals = m.away_goals THEN 'D'
            ELSE 'A' END,
       COALESCE(m.round_no > r.gameweek, TRUE)
FROM external_predictions e
```

```
JOIN matches m     ON m.match_id = e.match_id
JOIN seasons s     ON s.season_id = m.season_id
LEFT JOIN v_runs r ON r.run_label = e.run_label;

CREATE VIEW v_scored_predictions AS
SELECT a.*,
       POWER(p_home - (result = 'H')::INT, 2)
     + POWER(p_draw - (result = 'D')::INT, 2)
     + POWER(p_away - (result = 'A')::INT, 2) AS brier,
       0.5 * (POWER(p_home - (result = 'H')::INT, 2)
            + POWER(p_home + p_draw - (result IN ('H', 'D'))::INT, 2)) AS
rps,
       ((CASE WHEN p_home >= p_draw AND p_home >= p_away THEN 'H'
              WHEN p_away >= p_draw THEN 'A'
              ELSE 'D' END) = result) AS fav_correct
FROM v_all_predictions a
WHERE result IS NOT NULL AND out_of_sample;

CREATE VIEW v_scores AS
SELECT model, season, run_label,
       COUNT(*) AS matches,
       ROUND(AVG(brier), 4) AS brier,
       ROUND(AVG(rps), 4) AS rps,
       ROUND(AVG(fav_correct::INT), 3) AS pct_fav_correct
FROM v_scored_predictions
GROUP BY model, season, run_label;

CREATE VIEW v_variant_calibration AS
SELECT model, season, run_label,
       FLOOR(p_home * 10) / 10 AS band_start,
       (FLOOR(p_home * 10) * 10)::INT || '-' || (FLOOR(p_home * 10) * 10 +
10)::INT || '%' AS band,
       COUNT(*) AS matches,
       ROUND(AVG(p_home), 3) AS avg_predicted,
       ROUND(AVG((result = 'H')::INT), 3) AS actual_rate,
       ROUND(AVG((result = 'H')::INT) - AVG(p_home), 3) AS gap
FROM v_scored_predictions
GROUP BY model, season, run_label, FLOOR(p_home * 10);
```

Expect: a few notices (e.g. *view "v_rps" does not exist, skipping*), then *CREATE VIEW … Query returned successfully*.

If it stops with "cannot drop view … because other objects depend on it", you've built another view on top of one of these. The message names it; add that name to the `DROP VIEW` list and run again.

New ideas in this script, explained:

- **RPS (Ranked Probability Score)** treats the outcomes as ordered (home win, draw, away win), so predicting a draw when the home team wins is penalised less than predicting an away win. Formula for one match: ½ × [(P(home) − home won)² + (P(home or draw) − home won or drew)²].
- **`v_variant_ratings`** applies each variant's settings in order: blending, then the promoted-team regression (pre-season runs only), then the xG mix. With the `base` variant all switches are off, so the ratings are exactly your run ratings.
- **`v_match_predictions`** now picks the **adopted** variant (at the moment `base`), so `v_brier`, `v_calibration` and your Model scorecard work as before.

Step 0.3: Check nothing changed

```
SELECT s.run_label, s.brier AS new_brier, b.brier_model AS scorecard_brier,
s.rps
FROM v_scores s
JOIN v_brier b ON b.run_label = s.run_label
WHERE s.model = 'base'
ORDER BY s.run_label;
```

Expect: `new_brier` and `scorecard_brier` identical on every row (e.g. 0.6420 for 2025-26 preseason, 0.6118 for after GW22, 0.6228 for 2026-27 preseason), plus a new RPS column. If they match, the foundation is reproducing your model exactly.

This query can take 5–20 seconds, because it predicts every fixture for every run. That's normal.

Step 0.4: Add the Model comparison page to Excel

Your dashboard already has this page built: the **Model comparison** sheet, with two empty landing sheets, **Data_scores** and **Data_variant_calibration**. You only need to load the two views into the landing sheets, and the page fills itself in.

Load v_scores:

1. Close any other copy of the dashboard, then open `Prem_Dashboard.xlsx`.
2. **Data → Get Data → From Other Sources → From ODBC → prem_sims → OK**.
3. In the Navigator, click **v_scores**, then the arrow next to **Load → Load To…**
4. Choose **Table → Existing worksheet →** type `=Data_scores!$A$1` **→ OK**.

Load v_variant_calibration: repeat steps 2–4, choosing **v_variant_calibration** and typing `=Data_variant_calibration!$A$1`.

Turn off background refresh for both: Data → Queries & Connections → right-click each → **Properties** → untick **Enable background refresh** → **OK**.

Why cell A1 matters: the Model comparison formulas read the run label, model name and scores from fixed columns starting at A. Loading anywhere else leaves the page blank.

Using the Model comparison sheet:

- **C4:** pick a run. `2025-26 preseason` is the biggest test (380 matches).
- **Ranked table and RPS chart (top):** every variant that has scores for that run, best (lowest RPS) first. **RPS vs base** shows the change against your original model: negative means better. Right now it shows only `base`. Each upgrade adds a row.
- **Model list (K8:K17):** the variant names the page looks for. They match the names used in this guide. If you create a variant with a different name, type it into an empty yellow cell there.
- **Calibration (bottom):** pick two models in **C22** and **E22**. Each bar is actual minus predicted home-win rate for a band. A bar **above** zero means home teams won more often than predicted. You should see mostly positive bars for `base`: the missing home advantage that Upgrade 1 fixes.

Press **Ctrl+S**.

Upgrade 1: Home advantage

The problem: your model uses the same formula for both teams, so it treats every match as if played on neutral ground. Your calibration chart shows home teams winning far more often than predicted (e.g. 54% predicted, 67% actual).

The fix: multiply expected home goals by a **home factor** and expected away goals by an **away factor**, both measured from real results.

What I found when I tested it on your data: on 2025-26 gameweeks 1–22 (220 matches), home advantage improved the pre-season run's Brier from **0.634 to 0.620** and RPS from **0.218 to 0.211**. On the 46 matches of 2026-27 played so far it was slightly worse, because this season's home teams have barely outperformed away teams (18 home wins to 16 away). That's too few matches to trust, so judge on the full 2025-26 season.

Step 1.1: Measure the factors

In a pgAdmin Query Tool on prem_sims, run:

```
SELECT COUNT(*) AS matches,
       ROUND(AVG(home_goals), 3) AS avg_home_goals,
       ROUND(AVG(away_goals), 3) AS avg_away_goals,
       ROUND(AVG(home_goals) / ((AVG(home_goals) + AVG(away_goals)) / 2), 3)
AS home_factor,
       ROUND(AVG(away_goals) / ((AVG(home_goals) + AVG(away_goals)) / 2), 3)
AS away_factor
FROM matches m JOIN seasons s USING (season_id)
WHERE s.label = '2025-26' AND m.home_goals IS NOT NULL;
```

Expect: about **380** matches, home factor about **1.11** and away factor about **0.89**. Write your two numbers down; they're used below.

Why one full season: a whole season has every team playing 19 home and 19 away games, so it's balanced. Partial seasons can be skewed by who happened to play at home.

Step 1.2: Add the variant

Replace the two numbers with yours from Step 1.1:

```
INSERT INTO model_variants (variant, description, home_factor, away_factor)
VALUES ('home_adv', 'Home advantage from 2025-26 results', 1.113, 0.887);
```

Expect: `INSERT 0 1`.

Step 1.3: Compare with the current model

```
SELECT run_label, model, matches, brier, rps, pct_fav_correct
FROM v_scores
WHERE model IN ('base', 'home_adv')
ORDER BY run_label, model;
```

How to read it: for each run, compare the two rows. Lower `brier` and `rps` is better. The most trustworthy row is **2025-26 preseason** (380 matches). Record the RPS before and after in your results log.

A note on fairness: the factors came from 2025-26 results, so the 2025-26 test is slightly flattering. Home advantage is very stable from season to season, though, so the effect is small. The 2026-27 rows are a completely clean test, but on far fewer matches.

Step 1.4: Check the calibration

In Excel, **Data → Refresh All**, then go to **Model comparison**. Chart A now shows a `home_adv` bar. In Chart B, the `home_adv` bars should be closer to zero than the `base` bars in the busy bands (20–60%).

Step 1.5: Adopt it (only if RPS improved on 2025-26 preseason)

In the database:

```
UPDATE model_variants SET adopted = (variant = 'home_adv');
```

Expect: `UPDATE 2`. This marks `home_adv` as adopted and every other variant as not adopted. Your **Model scorecard** (v_brier, v_calibration) now shows the home-advantage model after **Refresh All**.

In your 2026-27 model workbook, so future simulations use it:

1. Open `prem_simulation_26-27.xlsx`. Set **Formulas → Calculation Options → Manual** while editing.
2. Insert a new sheet called `Settings`. In **A1** type `Home factor` and in **B1** your home factor (e.g. `1.113`). In **A2** type `Away factor` and in **B2** your away factor (e.g. `0.887`).
3. Name the cells: click **B1** → in the **Name Box** (left of the formula bar) type `HomeAdv` → **Enter**. Click **B2** → type `AwayAdv` → **Enter**.
4. Go to **Fixture sims updated**. Select column **F** (home score) by clicking its letter.
5. Press **Ctrl+H**. **Find what:** `*Average)` · **Replace with:** `*Average*HomeAdv)` → **Replace All**. Excel says how many it replaced: it should be the number of unplayed fixtures, or 380 if you linked this sheet to Results with IF formulas.
6. Select column **G** (away score). **Ctrl+H** → **Find what:** `*Average)` · **Replace with:** `*Average*AwayAdv)` → **Replace All**.
7. **Check:** click an unplayed fixture's home score cell. The formula should read `…*Average*HomeAdv)/10000,RAND())`, and the away cell `…*Average*AwayAdv)…`.
8. **For next season's pre-season run:** do the same on **Fixtures Sims**, column G with `HomeAdv` and column H with `AwayAdv`, replacing `*averagegoals25_26)` (or whichever average name that sheet uses).
9. Set calculation back to **Partial**, press **F9**, and save.

Why selecting the column first matters: with a column selected, Find & Replace only changes that column. Otherwise home and away formulas would both get `HomeAdv`.

What changes in the title odds: very little. Every team plays 19 home and 19 away games, so home advantage mostly cancels out over a season. Its real benefit is in **individual match** predictions, which is exactly what the Brier score measures.

On the H2H sheet in your dashboard: the inputs are already there. In the **Home advantage** box (K3:L5), **L4** is the home factor and **L5** the away factor, both set to **1** (off). Expected goals in **I12** and **I13** already multiply by them.

1. Type your home factor in **L4** (e.g. `1.113`) and your away factor in **L5** (e.g. `0.887`).

2. Check: for Manchester City v Brighton, the home-win % should rise and the away-win % fall compared with before.
3. To switch it off again, set both back to `1`.

Step 1.6: Once a season

At the start of each season, rerun Step 1.1 with the season that has just finished, then update the factors in three places: the `home_adv` row (`UPDATE model_variants SET home_factor = 1.10, away_factor = 0.90 WHERE variant = 'home_adv';`), the **Settings** sheet, and **H2H** L5:L6.

Upgrade 2: Blend ratings by games played

The problem: after a few games, a team's rating is based on very little data. After 5 games, Brighton's attack rating was 1.95 (nearly double average), which gave them a 56% title chance.

The fix: blend each team's pre-season rating with its current rating, weighted by games played:

`blended = (pre-season rating × k + current rating × games played) ÷ (k + games played)`

`k` means "treat pre-season knowledge as worth k games". With k = 10: after 5 games, pre-season counts double; after 30 games, this season counts three times as much.

Step 2.1: Add three test variants

These build on your home-advantage settings, so blending is the only difference being tested:

```
INSERT INTO model_variants (variant, description, home_factor, away_factor,
blend_k)
SELECT 'blend_' || w.k, 'Home advantage + blend weight ' || w.k,
       v.home_factor, v.away_factor, w.k
FROM model_variants v
CROSS JOIN (VALUES (5), (10), (15)) AS w(k)
WHERE v.variant = 'home_adv';
```

Expect: `INSERT 0 3`.

Step 2.2: Check the blending works

```
SELECT vr.variant, t.team_name, ROUND(vr.att, 3) AS att, ROUND(vr.def, 3) AS
def
FROM v_variant_ratings vr
JOIN teams t  USING (team_id)
JOIN v_runs r USING (run_id)
WHERE r.run_label = '2026-27 after GW5' AND t.team_name = 'Brighton'
ORDER BY vr.variant;
```

Expect: `home_adv` shows Brighton's attack at about **1.95** (unchanged), `blend_10` about **1.32** ((1.00 × 10 + 1.95 × 5) ÷ 15), `blend_5` higher and `blend_15` lower. Pre-season runs are never blended, because there's nothing to blend with.

Step 2.3: Compare

```
SELECT run_label, model, matches, brier, rps
FROM v_scores
WHERE model IN ('home_adv', 'blend_5', 'blend_10', 'blend_15')
  AND run_label LIKE '%after GW%'
ORDER BY run_label, rps;
```

The best `k` is at the top for each run. Record it in your results log.

Expect small differences on `2025-26 after GW22`. By gameweek 22 every team has 22 games, so blending changes ratings only a little. The big effect is early in the season, and at the moment your only early run (`2026-27 after GW5`) has no scored matches yet. Step 2.4 fixes that.

Step 2.4 (recommended): Create early-season test runs for 2025-26

To test blending properly you need runs made early in a season, such as *after GW5* and *after GW10*. You can recreate them for 2025-26, a season that has already finished.

For each gameweek you want (e.g. 5, then 10):

1. Make a **copy** of `prem_simulation_25-26.xlsx`, e.g. `prem_25-26_backtest.xlsx`. Never do this in the original.
2. In the copy, on **Results**, filter **Round Number** to show rounds **greater than** your gameweek, select those rows' score cells (Home Score and Away Score), press **Delete**, then clear the filter.
3. On **Fixture sims updated**, check those fixtures now simulate (they need the `IF(Results!…` formulas, or the `BINOM.INV` formula, rather than typed scores). If they hold typed scores, copy the formula down from an unplayed fixture.

4. Press **F9**, then export **ratings.csv** and **sims.csv** exactly as in your weekly guide, Stage B (B2 and B3), with the label `2025-26 after GW5`.
5. In pgAdmin, load **only these two files**: follow weekly guide Stage C but **skip results.csv, C4's results count and C7**. Never load results from the backtest copy, or the real 2025-26 scores would be blanked out.
6. Repeat for `2025-26 after GW10`.

Then rerun Step 2.3. You now have early-season runs with hundreds of scorable matches each (GW6–38 and GW11–38), which is a far stronger test.

Why these extra runs are safe for your dashboard: each has its own gameweek (5, 10), so your charts just gain extra 2025-26 columns. Nothing is doubled.

Step 2.5: Adopt the best k in your Excel model

Why not in SQL? From now on your Excel model saves *already-blended* ratings for each new run, so if SQL blended them again they'd be blended twice. The adopted SQL variant therefore stays `home_adv`, and the blending lives in Excel.

1. Open `prem_simulation_26-27.xlsx` and set calculation to **Manual** while editing.
2. On **Settings**, type `Blend weight` in **A3** and your best k (e.g. `10`) in **B3**. Name **B3** `BlendK` using the Name Box.
3. Go to **Team Strength updated**. In **D1** type `BLENDED ATT` and in **E1** `BLENDED DEF`.
4. In **D2** enter:

```
=LET(pre,XLOOKUP($A2,'Team Strength'!$A$2:$A$21,'Team
Strength'!$B$2:$B$21),
     g,XLOOKUP($A2,Table!$B$36:$B$55,Table!$C$36:$C$55),
     (pre*BlendK+B2*g)/(BlendK+g))
```

5. In **E2** enter:

```
=LET(pre,XLOOKUP($A2,'Team Strength'!$A$2:$A$21,'Team
Strength'!$C$2:$C$21),
     g,XLOOKUP($A2,Table!$B$36:$B$55,Table!$C$36:$C$55),
     (pre*BlendK+C2*g)/(BlendK+g))
```

6. Select **D2:E2** and fill down to row **21**.
7. **Check:** Brighton's **D** value should be about **1.32**, compared with **1.95** in column B.
8. **Formulas → Name Manager**: edit **Attack** to `='Team Strength updated'!$D$2:$D$21` and **Defence** to `='Team Strength updated'!$E$2:$E$21`.
9. Set calculation back to **Partial**, press **F9**, and save.

What the formula does: `pre` is the team's pre-season rating (reverted, column B or C of **Team Strength**), `B2` / `C2` is its current reverted rating, and `g` is games played from your current-season table. The current rating is reverted, to match exactly what was tested in SQL.

Change to your weekly routine: in weekly guide step **B2 (ratings.csv)**, copy columns **A, D, E** of Team Strength updated instead of A, B, C. Those are now the ratings your simulations use.

What you'll see: next week's run will have much steadier ratings, and Brighton's title chance should fall back towards Manchester City's.

Upgrade 3: Promoted-team regression

The problem: promoted clubs have no Premier League data, so their pre-season ratings are a judgement call. Your 2025-26 pre-season forecast was your weakest result.

The fix: use your **Historical Championship Data** (already loaded as `promoted_history`) to learn how a club's Championship ratings translate into its first Premier League season, using linear regression:

`predicted Premier League rating = slope × Championship rating + intercept`

The database calculates the slope and intercept for you (`v_promoted_fit`, built in Stage 0).

Step 3.1: Look at the regression

```
SELECT clubs,
       ROUND(att_slope::NUMERIC, 3) AS att_slope,
ROUND(att_intercept::NUMERIC, 3) AS att_intercept,
       ROUND(att_corr::NUMERIC, 3)  AS att_corr,
       ROUND(def_slope::NUMERIC, 3) AS def_slope,
ROUND(def_intercept::NUMERIC, 3) AS def_intercept,
       ROUND(def_corr::NUMERIC, 3)  AS def_corr
FROM v_promoted_fit;
```

How to read it:

- **clubs:** how many promoted clubs the regression learned from (about 42).
- **slope below 1:** Championship dominance only partly carries over. With a slope of 0.4, every extra 0.5 of Championship attack rating adds only 0.2 to the predicted Premier League attack rating.
- **corr (correlation):** how strongly Championship form predicts Premier League form, from 0 (not at all) to 1 (perfectly). Around 0.3–0.6 is typical: useful, but far from certain.

Step 3.2: See how well it would have predicted past clubs

Create a check view, then look at it:

```
CREATE OR REPLACE VIEW v_promoted_check AS
SELECT t.team_name, p.season_label,
       p.champ_att, p.epl_att,
       ROUND((f.att_slope * p.champ_att + f.att_intercept)::NUMERIC, 3) AS
predicted_epl_att,
       p.champ_def, p.epl_def,
       ROUND((f.def_slope * p.champ_def + f.def_intercept)::NUMERIC, 3) AS
predicted_epl_def
FROM promoted_history p
JOIN teams t USING (team_id)
CROSS JOIN v_promoted_fit f;

SELECT * FROM v_promoted_check ORDER BY season_label DESC;
```

Save this in a new tab as `sql\12_promoted.sql`.

Chart: predicted vs actual promoted-team attack

1. In Excel, load **v_promoted_check** (Data → Get Data → From ODBC → prem_sims → v_promoted_check → **Load To… → Table → New worksheet**). Rename the sheet `Data_promoted`.
2. Click an empty cell → **Insert → Scatter**. Right-click the blank chart → **Select Data → Add**: Series name `Attack`, **X values** = the `predicted_epl_att` column, **Y values** = the `epl_att` column (select the cells under each header) → **OK**.
3. Add a diagonal: **Add** again, name `Perfect prediction`, X values `={0.6,1.4}`, Y values `={0.6,1.4}`. Right-click that series → **Change Series Chart Type → Scatter with Straight Lines**.
4. Title: `Promoted clubs: predicted vs actual Premier League attack`.

Points near the line were predicted well. A wide cloud means promotion is hard to predict, which is why the next step tests it rather than assuming it helps.

Step 3.3: Find out what `season_label` means in your history

```
SELECT t.team_name, p.season_label, p.champ_att, p.epl_att
FROM promoted_history p JOIN teams t USING (team_id)
WHERE t.team_name IN ('Leeds United', 'Burnley', 'Sunderland')
ORDER BY p.season_label DESC;
```

These three were promoted into the 2025-26 Premier League. Note which `season_label` their latest rows have: it tells you whether your sheet labels a promotion by the **Championship season** (e.g. `2024-25`) or the **Premier League season** (`2025-26`). You need this in the next step.

Step 3.4: Enter the Championship ratings to test with

`promoted_inputs` holds the Championship ratings of each season's promoted clubs, labelled with the **Premier League season they're promoted into**.

2025-26 (Leeds, Burnley, Sunderland): copy their rows from your history. Replace `'2024-25'` with the label you found in Step 3.3:

```
INSERT INTO promoted_inputs (season_label, team_id, champ_att, champ_def)
SELECT '2025-26', p.team_id, p.champ_att, p.champ_def
FROM promoted_history p JOIN teams t USING (team_id)
WHERE t.team_name IN ('Leeds United', 'Burnley', 'Sunderland')
  AND p.season_label = '2024-25';
```

Expect: `INSERT 0 3`. If you get 0, check the label and team spellings from Step 3.3.

2026-27 (Coventry City, Hull City, Ipswich Town): work out their 2025-26 Championship ratings the same way as your Historical Championship Data sheet. If that sheet uses goals per game relative to the league average:

1. Get the 2025-26 Championship final table (BBC Sport or FBref) into a blank Excel sheet with columns **Team, Played, GF, GA**.
2. League average: `=SUM(GF column)/SUM(Played column)`.
3. For each promoted club: **ATT** `=(GF/Played)/league average`, **DEF** `=(GA/Played)/league average`.

Then insert them, replacing the numbers with yours:

```
INSERT INTO promoted_inputs (season_label, team_id, champ_att, champ_def)
SELECT '2026-27', team_id, 1.40, 0.80 FROM teams WHERE team_name = 'Coventry
City'
UNION ALL
SELECT '2026-27', team_id, 1.30, 0.85 FROM teams WHERE team_name = 'Hull
City'
UNION ALL
SELECT '2026-27', team_id, 1.25, 0.90 FROM teams WHERE team_name = 'Ipswich
Town';
```

Expect: `INSERT 0 3`.

A note on fairness: if Leeds, Burnley and Sunderland's 2025-26 Premier League ratings are already in your history, the regression has partly "seen" the answer for the 2025-26 test. With around 42 clubs in the regression the effect is small, but the 2026-27 test is the clean one.

Step 3.5: Add the variant

```
INSERT INTO model_variants (variant, description, home_factor, away_factor,
promoted_reg)
SELECT 'promoted_reg', 'Home advantage + regression ratings for promoted
clubs',
       home_factor, away_factor, TRUE
FROM model_variants WHERE variant = 'home_adv';
```

Check the promoted clubs got new ratings:

```
SELECT r.run_label, t.team_name, vr.variant, ROUND(vr.att, 3) AS att,
ROUND(vr.def, 3) AS def
FROM v_variant_ratings vr
JOIN teams t  USING (team_id)
JOIN v_runs r USING (run_id)
WHERE r.gameweek = 0
  AND vr.variant IN ('home_adv', 'promoted_reg')
  AND t.team_name IN ('Leeds United', 'Burnley', 'Sunderland', 'Coventry
City', 'Hull City', 'Ipswich Town')
ORDER BY r.run_label, t.team_name, vr.variant;
```

For each promoted club, the two rows should differ. Everyone else's ratings are unchanged.

Step 3.6: Compare

All matches in pre-season runs:

```
SELECT run_label, model, matches, brier, rps
FROM v_scores
WHERE model IN ('home_adv', 'promoted_reg') AND run_label LIKE '%preseason'
ORDER BY run_label, rps;
```

Only matches involving promoted clubs (a sharper test, since only those predictions change):

```
SELECT sp.run_label, sp.model, COUNT(*) AS matches, ROUND(AVG(sp.rps), 4) AS
rps
FROM v_scored_predictions sp
JOIN matches m          ON m.match_id = sp.match_id
JOIN seasons s          ON s.season_id = m.season_id
JOIN promoted_inputs pi ON pi.season_label = s.label
                       AND pi.team_id IN (m.home_team_id, m.away_team_id)
WHERE sp.model IN ('home_adv', 'promoted_reg') AND sp.run_label LIKE
'%preseason'
GROUP BY sp.run_label, sp.model
ORDER BY sp.run_label, rps;
```

(A match between two promoted clubs is counted twice here. With three promoted clubs that's only 6 matches a season, so it barely matters.)

Step 3.7: Adopt it (only if it helped)

Promoted clubs only matter at pre-season, so you apply this **when you build next season's model**:

1. Get the slope and intercept from Step 3.1 (rerun it after adding the latest season's promoted clubs to `promoted_history`).
2. In next season's **Team Strength** sheet, for each promoted club, overwrite the rating columns your simulation uses: **attack** = `att_slope × Championship ATT + att_intercept`, **defence** = `def_slope × Championship DEF + def_intercept`. Type them as formulas so you can see the working.
3. Add a note beside them, e.g. `Regression from v_promoted_fit, 42 clubs`.

In SQL the adopted variant stays `home_adv`: your saved pre-season ratings will already contain the regression values, so applying it again isn't needed.

Upgrade 4: Bookmaker benchmark

Why: beating the no-skill baseline proves your model has *some* skill. Comparing it with the bookmakers shows how good it is. Bookmaker odds include team news, injuries and huge amounts of money, so they're very hard to beat; getting close is a real achievement.

How: turn each match's odds into probabilities, store them as an external "model" called `bookmaker`, and score them on exactly the same matches as your model.

Step 4.1: Download the odds

1. Go to **football-data.co.uk** → **Data Files** → **England** → find **Premier League** under **Season 2025/2026**.

2. Download the file (`E0.csv`) and save it as `odds_2025-26.csv` .
3. Do the same for **Season 2026/2027** (`odds_2026-27.csv`). The site updates the current season during the season.

Step 4.2: Prepare odds.csv in Excel

1. Open `odds_2025-26.csv` . Find the columns **HomeTeam**, **AwayTeam**, **AvgH**, **AvgD** and **AvgA** (the average odds across bookmakers for home win, draw and away win). If there are no Avg columns, use **B365H**, **B365D** and **B365A** instead.
2. Copy those five columns (Ctrl+click each column letter, then Ctrl+C) → **Ctrl+N** → paste into **B1** of the new workbook.
3. In **A1** type `season` ; in **A2** type `2025-26` and fill it down to the last row.
4. Rename the headers **B1:F1** to `home_team` , `away_team` , `odds_h` , `odds_d` , `odds_a` .
5. Open `odds_2026-27.csv` , copy the same five columns **without the header row**, and paste them under the 2025-26 rows starting in column **B**. Put `2026-27` in column A for those rows.
6. Delete any columns to the right of F (click column G, **Ctrl+Shift+→**, right-click → **Delete**).
7. **Save As → CSV UTF-8** → `odds.csv` . Close it.

Step 4.3: Load into the database

In a pgAdmin Query Tool on prem_sims (save this tab as `sql\13_bookmaker.sql`):

```
CREATE TABLE IF NOT EXISTS stg_odds (
  season TEXT, home_team TEXT, away_team TEXT,
  odds_h TEXT, odds_d TEXT, odds_a TEXT
);
TRUNCATE stg_odds;
```

Then import `odds.csv` into **stg_odds** (right-click the table → **Import/Export Data** → Import → csv, UTF8, Header on).

Find team names the database doesn't know:

```
SELECT DISTINCT n AS unknown_name
FROM (SELECT TRIM(home_team) AS n FROM stg_odds
      UNION SELECT TRIM(away_team) FROM stg_odds) x
WHERE n NOT IN (SELECT alias FROM team_aliases);
```

Football-data uses short names such as `Man United` , `Nott'm Forest` and `Hull` . For each unknown name, add an alias pointing at the right club, for example:

```
INSERT INTO team_aliases (alias, team_id)
SELECT 'Man United', team_id FROM teams WHERE team_name = 'Manchester
United';
INSERT INTO team_aliases (alias, team_id)
SELECT 'Hull', team_id FROM teams WHERE team_name = 'Hull City';
```

Rerun the check until it returns no rows.

Turn the odds into probabilities and store them:

```
INSERT INTO external_predictions (source, run_label, match_id, p_home,
p_draw, p_away)
SELECT 'bookmaker', 'odds', m.match_id,
       ROUND((1 / o.h) / (1 / o.h + 1 / o.d + 1 / o.a), 4),
       ROUND((1 / o.d) / (1 / o.h + 1 / o.d + 1 / o.a), 4),
       ROUND((1 / o.a) / (1 / o.h + 1 / o.d + 1 / o.a), 4)
FROM (SELECT season, TRIM(home_team) AS home_team, TRIM(away_team) AS
away_team,
             NULLIF(odds_h, '')::NUMERIC AS h,
             NULLIF(odds_d, '')::NUMERIC AS d,
             NULLIF(odds_a, '')::NUMERIC AS a
      FROM stg_odds) o
JOIN seasons s      ON s.label = o.season
JOIN team_aliases h ON h.alias = o.home_team
JOIN team_aliases a ON a.alias = o.away_team
JOIN matches m      ON m.season_id = s.season_id
                   AND m.home_team_id = h.team_id AND m.away_team_id =
a.team_id
WHERE o.h IS NOT NULL AND o.d IS NOT NULL AND o.a IS NOT NULL
ON CONFLICT (source, run_label, match_id)
DO UPDATE SET p_home = EXCLUDED.p_home, p_draw = EXCLUDED.p_draw, p_away =
EXCLUDED.p_away;
```

Expect: `INSERT 0 n`, where n is the number of matches in your file (380 for 2025-26 plus the 2026-27 matches played so far).

Why divide by the total: bookmaker odds include a profit margin, so 1/odds for the three outcomes adds up to more than 1 (about 1.05). Dividing each by the total removes the margin, leaving fair probabilities that add up to 1.

Why `ON CONFLICT … DO UPDATE`*:* when you reload the file later in the season, existing matches are updated instead of causing a duplicate-key error.

Step 4.4: Compare on the same matches

Create a view that pairs every model prediction with the bookmaker's prediction for the same match:

```
CREATE OR REPLACE VIEW v_vs_bookmaker AS
SELECT sp.model, sp.season, sp.run_label,
       COUNT(*) AS matches,
       ROUND(AVG(sp.brier), 4) AS model_brier,
       ROUND(AVG(bk.brier), 4) AS bookmaker_brier,
       ROUND(AVG(sp.rps), 4)   AS model_rps,
       ROUND(AVG(bk.rps), 4)   AS bookmaker_rps
FROM v_scored_predictions sp
JOIN v_scored_predictions bk ON bk.match_id = sp.match_id AND bk.model =
'bookmaker'
WHERE sp.model <> 'bookmaker'
GROUP BY sp.model, sp.season, sp.run_label;

SELECT * FROM v_vs_bookmaker ORDER BY run_label, model_rps;
```

How to read it: each row compares one model with the bookmakers on **identical** matches. If `model_rps` is close to `bookmaker_rps` (within about 0.005), your model is doing very well.

Keep in mind: a pre-season forecast made in August is up against odds set the day before each match, with full team news, so the gap will be largest for pre-season runs. The fairest comparison is your **latest** run against the bookmakers on the following gameweek's matches.

Step 4.5: Chart it in Excel

1. Load **v_vs_bookmaker** into Excel (Data → Get Data → From ODBC → prem_sims → v_vs_bookmaker → **Load To… → Table → New worksheet**). Rename the sheet `Data_vs_bookmaker`, and turn off background refresh for the query.
2. Click inside the table → **Insert → PivotTable** → **Existing Worksheet** → a free cell on your **Model comparison** sheet.
3. **Filters:** `run_label`. **Rows:** `model`. **Values:** `model_rps` and `bookmaker_rps`.
4. **PivotChart → Bar → Clustered Bar**. Title: `Your models vs the bookmakers (RPS, lower is better)`.
5. Fix the axis as usual: right-click the model names → **Format Axis** → **Categories in reverse order** and **Horizontal axis crosses: At maximum category**.

Step 4.6: Keep it updated

Every few weeks, download the latest 2026-27 file, rebuild `odds.csv` (Step 4.2), then in pgAdmin run `TRUNCATE stg_odds;`, import `odds.csv`, and rerun the `INSERT INTO external_predictions …` block. Then **Refresh All** in Excel.

Upgrade 5: Expected goals (xG) ratings

Why: goals are rare and luck-driven. A team can win 1-0 three times while being outplayed. **Expected goals (xG)** measure the quality of chances each team creates and allows, so they reveal true strength from fewer games. Professional models rate teams on xG, or on a mix of goals and xG.

How: load match-by-match xG, build attack and defence ratings from it (with the same 25% reversion as your goal ratings), then test mixing them with your goal-based ratings.

Step 5.1: Get the xG data

1. On **fbref.com**, go to **Competitions → Premier League → 2025-2026 → Scores & Fixtures**. The table shows each match's **xG** for the home side (left of the score) and the away side (right of the score).
2. Above the table, click **Share & Export → Get table as CSV (for Excel)**. Copy the text that appears, paste it into **Notepad**, and save it as `fbref_2025-26.csv`.
3. Repeat for **2026-2027**.

If FBref doesn't show xG for a season, **understat.com** (League → EPL → season) lists match xG too. Copy its match table into Excel instead.

Step 5.2: Prepare xg.csv in Excel

1. Open `fbref_2025-26.csv` in Excel. Keep only the columns **Home**, the first **xG**, the second **xG** and **Away**.
2. Delete rows without xG (unplayed matches and any repeated header rows): filter the xG column for blanks, delete those rows, then clear the filter.
3. Arrange the columns as: **A** `season` (fill with `2025-26`), **B** `home_team`, **C** `away_team`, **D** `xg_home`, **E** `xg_away`. Note that Away moves before the two xG columns.
4. Add the 2026-27 rows underneath in the same layout, with `2026-27` in column A.
5. Delete any columns to the right of E, then **Save As → CSV UTF-8** → `xg.csv`.

Step 5.3: Load into the database

In pgAdmin (save this tab as `sql\14_xg.sql`):

```
CREATE TABLE IF NOT EXISTS stg_xg (
  season TEXT, home_team TEXT, away_team TEXT, xg_home TEXT, xg_away TEXT
);
TRUNCATE stg_xg;
```

Import `xg.csv` into **stg_xg** (csv, UTF8, Header on). Then check for unknown team names:

```
SELECT DISTINCT n AS unknown_name
FROM (SELECT TRIM(home_team) AS n FROM stg_xg
      UNION SELECT TRIM(away_team) FROM stg_xg) x
WHERE n NOT IN (SELECT alias FROM team_aliases);
```

Add an alias for each name it lists (as in Step 4.3), then store the xG:

```
INSERT INTO match_xg (match_id, xg_home, xg_away)
SELECT m.match_id, NULLIF(x.xg_home, '')::NUMERIC, NULLIF(x.xg_away,
'')::NUMERIC
FROM stg_xg x
JOIN seasons s      ON s.label = x.season
JOIN team_aliases h ON h.alias = TRIM(x.home_team)
JOIN team_aliases a ON a.alias = TRIM(x.away_team)
JOIN matches m      ON m.season_id = s.season_id
                   AND m.home_team_id = h.team_id AND m.away_team_id =
a.team_id
WHERE NULLIF(x.xg_home, '') IS NOT NULL
ON CONFLICT (match_id)
DO UPDATE SET xg_home = EXCLUDED.xg_home, xg_away = EXCLUDED.xg_away;
```

Check every played match has xG:

```
SELECT s.label, COUNT(*) AS played, COUNT(x.match_id) AS with_xg
FROM matches m
JOIN seasons s ON s.season_id = m.season_id
LEFT JOIN match_xg x ON x.match_id = m.match_id
WHERE m.home_goals IS NOT NULL
GROUP BY s.label;
```

Expect: `played` and `with_xg` equal for each season. If not, some names didn't match, so recheck the unknown-names query.

Step 5.4: Who is over- and under-performing their xG?

```
CREATE OR REPLACE VIEW v_team_xg AS
WITH tm AS (
  SELECT m.season_id, m.home_team_id AS team_id, m.home_goals AS gf,
m.away_goals AS ga,
         x.xg_home AS xgf, x.xg_away AS xga
  FROM matches m JOIN match_xg x ON x.match_id = m.match_id
  WHERE m.home_goals IS NOT NULL
  UNION ALL
  SELECT m.season_id, m.away_team_id, m.away_goals, m.home_goals, x.xg_away,
x.xg_home
  FROM matches m JOIN match_xg x ON x.match_id = m.match_id
  WHERE m.home_goals IS NOT NULL
)
SELECT s.label AS season, t.team_name,
       COUNT(*) AS played,
       SUM(tm.gf) AS gf, ROUND(SUM(tm.xgf), 1) AS xgf,
       SUM(tm.ga) AS ga, ROUND(SUM(tm.xga), 1) AS xga,
       ROUND(SUM(tm.gf) - SUM(tm.xgf), 1) AS goals_above_xg,
       ROUND(SUM(tm.xga) - SUM(tm.ga), 1) AS conceded_below_xg
FROM tm
JOIN teams t   ON t.team_id = tm.team_id
JOIN seasons s ON s.season_id = tm.season_id
GROUP BY s.label, t.team_name;

SELECT * FROM v_team_xg WHERE season = '2026-27' ORDER BY goals_above_xg
DESC;
```

A big positive `goals_above_xg` means a team is scoring more than its chances deserve. That usually doesn't last, and it's exactly the kind of team your igoal-based ratings overrate.

Chart: luck vs quality

1. Load **v_team_xg** into Excel (Load To… → Table → New worksheet), rename the sheet `Data_team_xg`, and turn off background refresh.
2. **Insert → PivotTable** on a new sheet called `xG analysis`. **Filters:** `season` (2026-27). **Rows:** `team_name`. **Values:** `goals_above_xg`.
3. Sort descending by the value, then **PivotChart → Bar → Clustered Bar**.
4. Title: `Goals scored above (+) or below (−) expected goals`.

Step 5.5: Test xG-based ratings

The xG variants build on your best model so far, so the only new thing being tested is xG. Replace `'blend_10'` with the variant you kept in Upgrade 2 (or `'home_adv'` if blending didn't help):

```
INSERT INTO model_variants (variant, description, home_factor, away_factor,
blend_k, xg_weight)
SELECT 'xg_' || (w.x * 100)::INT, 'Best model so far + ' || (w.x * 100)::INT
|| '% xG',
       v.home_factor, v.away_factor, v.blend_k, w.x
FROM model_variants v
CROSS JOIN (VALUES (0.5), (1.0)) AS w(x)
WHERE v.variant = 'blend_10';
```

This creates `xg_50` (half goals, half xG) and `xg_100` (xG only). Compare them on the runs made after some games had been played:

```
SELECT run_label, model, matches, brier, rps
FROM v_scores
WHERE model IN ('blend_10', 'xg_50', 'xg_100')
  AND run_label LIKE '%after GW%'
ORDER BY run_label, rps;
```

(Change `'blend_10'` to your kept variant here too.) Pre-season runs aren't affected, because xG ratings are built only from the current season's matches before each run.

Step 5.6: Adopt it in your Excel model (only if it helped)

Your model workbook needs each team's xG totals. The easiest way is to connect it to the same database view:

1. Open `prem_simulation_26-27.xlsx` → **Data → Get Data → From Other Sources → From ODBC → prem_sims → v_team_xg → Load To… → Table → New worksheet**. Rename the sheet `xG`. Its columns are A `season`, B `team_name`, C `played`, D `gf`, E `xgf`, F `ga`, G `xga`.
2. On **Settings**, type `xG weight` in **A4** and your best weight (`0.5` or `1`) in **B4**. Name **B4** `XgWeight`.
3. On **Team Strength updated**, add these headers: **H1** `xG ATT`, **I1** `xG DEF`, **J1** `FINAL ATT`, **K1** `FINAL DEF`.
4. **H2:**

```
=LET(xf,SUMIFS(xG!$E:$E,xG!$A:$A,"2026-27",xG!$B:$B,$A2),
     p,SUMIFS(xG!$C:$C,xG!$A:$A,"2026-27",xG!$B:$B,$A2),
     lg,SUMIFS(xG!$E:$E,xG!$A:$A,"2026-
27")/SUMIFS(xG!$C:$C,xG!$A:$A,"2026-27"),
     IF(p=0,1,1+((xf/p)/lg-1)*0.75))
```

5. **I2:** the same formula, but with `xG!$G:$G` (xG against) in place of `xG!$E:$E` on the **first** line only. The league average on the third line stays as `$E:$E`, because total xG for equals total xG against across the league.
6. **J2:** `=(1-XgWeight)*D2+XgWeight*H2` and **K2:** `=(1-XgWeight)*E2+XgWeight*I2`. (D and E are your blended ratings from Upgrade 2. If you didn't keep blending, use B and C instead.)
7. Select **H2:K2** and fill down to row **21**.
8. **Formulas → Name Manager:** point **Attack** at `='Team Strength updated'!$J$2:$J$21` and **Defence** at `='Team Strength updated'!$K$2:$K$21`.
9. Press **F9** and save.

Changes to your weekly routine:

- After loading results (weekly guide Stage C), also update xG: rebuild `xg.csv` from FBref, then in pgAdmin run `TRUNCATE stg_xg;`, import it, and rerun the `INSERT INTO match_xg …` block.
- In the model workbook, **Data → Refresh All** so the **xG** sheet updates **before** you press F9 in Stage A. So the new order each week is: load results into the database → load xG → refresh the model's xG sheet → F9 → export.
- In weekly guide step **B2**, export columns **A, J, K** of Team Strength updated as `ratings.csv`.

Upgrade 6: Dixon–Coles model in Python

What Dixon–Coles is: the standard academic football model (Dixon & Coles, 1997). Instead of calculating each team's rating from its own goals, it fits **all teams' attack and defence ratings at once**, plus home advantage, so a 4-0 win against the bottom side counts for less than against the top side. It also:

- **weights recent matches more** (time decay), so last season still counts, but less;
- **corrects low scores** (0-0, 1-0, 0-1 and 1-1 happen more often than the plain Poisson formula predicts), through a parameter called *rho*;
- in this version, **shrinks** teams with little data towards average, so a promoted club's first few games can't make it a title contender.

What the script does for each of your runs: fits the model using only matches played before that run, predicts every fixture of the season (stored in `external_predictions` and scored automatically by `v_scores`), simulates the rest of the season 10,000 times (`dc_sim_summary`), and saves the fitted ratings (`dc_team_ratings`).

What I found when I tested it on your results: it fits in under a second per run. It works best with plenty of history. Fitted on only 150 matches from one season, it did worse than the baseline, and shrinkage helped a lot (early-season Brier 0.684 → 0.649). For 2026-27 runs it uses all of 2025-26 as history. Expect it to compete with your model rather than automatically beat it; finding out which is better is the point of the test.

Step 6.1: Install Python (one time)

1. Go to **python.org → Downloads** and download the latest Python 3 for Windows.
2. Run the installer. **On the first screen, tick "Add python.exe to PATH"**, then click **Install Now**.
3. Check it worked: open **Command Prompt** (Start → type `cmd` → Enter) and type `python --version`. You should see e.g. `Python 3.13.x`.

Step 6.2: Install the libraries (one time)

In Command Prompt, type:

```
pip install pandas numpy scipy sqlalchemy psycopg2-binary
```

Wait for *Successfully installed ….*

Library	What it does in the script
pandas	Tables of matches and results
numpy	Fast maths for the 10,000 simulations
scipy	Finds the best-fitting ratings (optimisation)
sqlalchemy + psycopg2-binary	Connects Python to your PostgreSQL database

Step 6.3: Set up the script

1. Create a folder `python` inside your project folder (e.g. `Documents\Prem SQL Project\python`).
2. Save `dc_model.py` (attached in the chat) into it.

3. Open it in **Notepad** (right-click → Open with → Notepad). At the top, in the **CONFIG** section, replace `YOUR_PASSWORD` with your pgAdmin `postgres` password. Save.

The settings in CONFIG:

Setting	Default	Meaning
`XI`	0.0019	Time decay per day. 0.0019 means a match a year old counts about half as much as yesterday's
`RIDGE`	5	Shrinkage towards average. 0 = none; 3–10 worked best in my tests
`N_SIMS`	10000	Simulated seasons per run
`ONLY_RUN`	None	Set to e.g. `"2026-27 after GW6"` to fit just one run (weekly use)

Password safety: the password is stored in plain text, so don't share this file or upload it to GitHub without removing the password first.

Step 6.4: Run it

1. In Command Prompt, go to the folder, keeping the quotes:

```
cd "%USERPROFILE%\Documents\Prem SQL Project\python"
```

2. Run:

```
python dc_model.py
```

3. **Expect** one line per run, for example:
   - `2025-26 preseason: skipped: only 0 matches available before this run`. That's correct: your database has no 2024-25 matches to learn from.
   - `2026-27 preseason: fitted on 380 matches; home advantage x1.2…; no data for Coventry City, Hull City, Ipswich Town (set to bottom-3 average)`
   - `Done. In Excel: Data > Refresh All.`

If you see an error, check Troubleshooting at the end of this guide.

Step 6.5: Compare it with your model

```
SELECT run_label, model, matches, brier, rps
FROM v_scores
WHERE model IN ('dixon_coles', 'home_adv', 'blend_10', 'xg_50')
ORDER BY run_label, rps;
```

(Use the variant names you kept.) Also check it against the bookmakers:

```
SELECT * FROM v_vs_bookmaker WHERE model = 'dixon_coles' ORDER BY run_label;
```

Check the fitted ratings look sensible:

```
SELECT team_name, attack, defence, home_adv, rho
FROM dc_team_ratings
WHERE run_label = '2026-27 after GW5'
ORDER BY attack DESC;
```

The strongest attacks should be at the top. `home_adv` is usually 1.1–1.3, and `rho` slightly negative (about −0.05 to −0.15).

Step 6.6: Tune it

Try a few settings and keep the combination with the lowest RPS on your largest test sample:

1. In `dc_model.py`, change `RIDGE` (try 3, 5, 10), save, and run `python dc_model.py` again. Rerunning replaces the previous Dixon–Coles results.
2. Rerun the Step 6.5 query and note the RPS for each setting in your results log.
3. Then try `XI` (e.g. 0.001, 0.0019, 0.003) with your best `RIDGE`.

Step 6.7: Charts

Title odds: your model vs Dixon–Coles

```
CREATE OR REPLACE VIEW v_title_compare AS
SELECT s.season, s.run_label, s.team_name,
       s.title_pct      AS model_title_pct,      d.title_pct      AS
dc_title_pct,
       s.top4_pct       AS model_top4_pct,       d.top4_pct       AS
dc_top4_pct,
       s.relegation_pct AS model_relegation_pct, d.relegation_pct AS
dc_relegation_pct
FROM v_sim_summary s
JOIN dc_sim_summary d ON d.run_label = s.run_label AND d.team_name =
s.team_name;
```

1. Load **v_title_compare** into Excel (new worksheet, rename `Data_title_compare`, background refresh off).
2. **Insert → PivotTable** on a new sheet `Model vs Dixon-Coles`. **Filters:** `run_label` (your latest run). **Rows:** `team_name`. **Values:** `model_title_pct` and `dc_title_pct`.
3. Row Labels dropdown → **Value Filters → Greater Than** → `Sum of model_title_pct` > `2`.
4. **PivotChart → Bar → Clustered Bar**, fix the axis as usual, and title it `Title chances: Excel model vs Dixon-Coles`.
5. Copy the PivotTable and chart, and change the values to the relegation columns for a second chart.

Where the two models disagree strongly, look at why: it's often a team whose results flatter or understate it.

Dixon–Coles ratings map

1. Load the **dc_team_ratings** table (it appears in the ODBC navigator like the views). Rename the sheet `Data_dc_ratings`.
2. Build a small block with FILTER, as you did for the Predictions chart: in an empty sheet, `=FILTER(Data_dc_ratings!B2:D500, Data_dc_ratings!A2:A500="2026-27 after GW5")` (adjust the ranges to your table).
3. **Insert → Scatter**: X = defence, Y = attack. Title: `Dixon-Coles ratings (top-left = strong attack, strong defence)`.

Step 6.8: Weekly use

After loading each new run (weekly guide Stage C), set `ONLY_RUN = "2026-27 after GW6"` (your new label) in the script, run `python dc_model.py`, then **Refresh All** in Excel. To refit every run at once (e.g. after tuning), set `ONLY_RUN = None`.

Should Dixon–Coles replace your Excel model? Only if it beats your best variant on RPS over a large sample, ideally a whole season. Until then, use it as a second opinion: when the two agree, you can be more confident, and when they disagree, it's worth investigating.

After the upgrades: your weekly routine and CV

What changes in the weekly routine

Your weekly guide (Part 10 of the step-by-step plan) still applies. These are the additions, in the order you do them, depending on which upgrades you kept:

When	What to do	Needed if you kept
Before Stage A	In pgAdmin, load the new results first (Stage C2–C4 and C7 only), then load the latest xG (Step 5.3 insert). In the model workbook, **Data → Refresh All** so the **xG** sheet updates	Upgrade 5 (xG)
Stage A	Nothing new: home advantage and blending are built into the model's formulas	Upgrades 1, 2
Stage B2	Export **ratings.csv** from the columns the `Attack` and `Defence` names point to: A, D, E (blending) or A, J, K (xG)	Upgrades 2, 5
After Stage C	Run `python dc_model.py` with `ONLY_RUN` set to the new label	Upgrade 6
Every few weeks	Refresh the bookmaker odds (Step 4.6)	Upgrade 4
Stage E	**Refresh All**, then check the **Model comparison** sheet: the previous run's scores should have grown	All
Once a season	Recompute home factors (Step 1.6), redo the promoted-team ratings (Step 3.7), and check your blend weight still wins	Upgrades 1–3

If you kept xG, the order of the weekly routine becomes: load results into the database → load xG → refresh the model's xG sheet → Stage A (F9) → Stage B → load the new run (C8–C10) → Dixon–Coles → dashboard. When results are loaded early like this, skip C7 in Stage C, because the scores are already in.

What to put on your CV

Use your own numbers from the results log. For example:

- *"Built a model-testing framework in PostgreSQL that re-predicts every historical fixture under alternative model settings and scores them with the Brier score and Ranked Probability Score."*
- *"Improved match-prediction accuracy by X% (RPS) by adding home advantage and games-played rating shrinkage, validated out-of-sample on 380+ matches."*
- *"Implemented a Dixon–Coles model in Python (time-decayed maximum likelihood, low-score correction) and benchmarked it against bookmaker odds."*

Be ready to explain in an interview: why you test out-of-sample, what RPS measures and why it suits football, why you change one thing at a time, and one upgrade that *didn't* help, and what you learned from it. Showing that you tested something and rejected it is as valuable as an improvement.

Troubleshooting

Problem	Likely cause	Fix
Stage 0 stops with *cannot drop view … because other objects depend on it*	A view you created depends on one being rebuilt	Add the named view to the `DROP VIEW` list at the top of section 2 and run again. Afterwards, rerun the script that created it (e.g. `12_promoted.sql`, `13_bookmaker.sql`, `14_xg.sql`)
Rerunning your Part 11 views script fails because of `v_title_compare`	`v_title_compare` depends on `v_sim_summary`	Run `DROP VIEW IF EXISTS v_title_compare;` first, then rerun Part 11, then recreate `v_title_compare` (Step 6.7)
Step 0.3: `new_brier` differs from `scorecard_brier`	`base` isn't the adopted variant, or the base row was changed	`UPDATE model_variants SET adopted = (variant = 'base');` and check the base row has home/away factor 1, blend_k empty, xg_weight 0

Problem	Likely cause	Fix
Queries take a long time (30 seconds or more)	Every variant re-predicts every fixture	Delete variants you've finished testing: `DELETE FROM model_variants WHERE variant IN ('blend_5', 'blend_15');` (never delete the adopted one)
`duplicate key value violates unique constraint "model_variants_pkey"`	That variant name already exists	Use a new name, or update the existing row with `UPDATE model_variants SET … WHERE variant = '…';`
`duplicate key … "team_aliases_pkey"` when adding an alias	The alias already exists	Skip it; it's already mapped
Bookmaker or xG insert says `INSERT 0 0`	Season labels or team names don't match	Rerun the unknown-names check; make sure column A says exactly `2025-26` or `2026-27`
Excel Model comparison chart shows nothing new	The query didn't refresh	Data → Queries & Connections → right-click v_scores → Refresh, then Refresh All
`python` is not recognised	Python isn't on the PATH	Reinstall Python and tick **Add python.exe to PATH**, then open a **new** Command Prompt
`ModuleNotFoundError: No module named …`	A library isn't installed	Rerun `pip install pandas numpy scipy sqlalchemy psycopg2-binary`
`password authentication failed`	Wrong password in `DB_URL`	Fix it in the CONFIG section. If your password contains `@`, `:` or `/`, replace them with `%40`, `%3A` and `%2F`

Problem	Likely cause	Fix
`relation "external_predictions" does not exist`	Stage 0 wasn't run	Run the Stage 0 script first
The script prints *WARNING: optimiser did not fully converge*	Not enough data, or extreme results	Increase `RIDGE` (e.g. to 10) and rerun
Dixon–Coles gives a promoted club strange odds	Few matches for that club	Increase `RIDGE`, or wait a few more gameweeks
Excel formulas show `#NAME?` for `HomeAdv`, `BlendK` or `XgWeight`	The named cell wasn't created	Click the setting's cell on **Settings**, type the exact name in the Name Box, and press Enter
After changing `Attack` / `Defence`, the simulations give odd results	The new columns aren't filled for all 20 teams	Check D2:E21 (or J2:K21) have numbers in every row, with no blanks or `#N/A`

If you get stuck, copy the exact error message (from pgAdmin's bottom panel, Command Prompt or Excel) and paste it into the chat.